

Taller 2 — Evolucionando mi catálogo React

Objetivo

Al finalizar el aprendizaje deberá poder:

- recuperar su proyecto desde GitHub;
- crear nuevos componentes;
- comprender mejor las props;
- usar eventos como onClick y onChange;
- comprender qué es el estado;
- utilizar useState();
- realizar búsquedas y filtros;
- modificar visualmente la interfaz según los datos;
- continuar documentando el proyecto mediante Git y GitHub.

1. Punto de partida

Primero todos deben recuperar **el trabajo de ayer**.

Abrir la carpeta:

```
tienda-react
```

Luego en terminal:

```
git status
```

Si trabajan desde el mismo computador:

```
git pull
```

Después:

```
npm install
```

Solo si hace falta node_modules.

Y ejecutar:

```
npm run dev
```

Control

Antes de continuar deben comprobar que se vea:

- catálogo;
- productos;
- precio;
- stock;
- cantidad disponible;
- valor del inventario.

No se avanza si el proyecto anterior no funciona.

2. Mejorar ProductoCard

Ayer tenían algo similar a:

```
01 JavaScript

function ProductoCard({ producto }) {

  const estado =
    producto.stock > 0
    ? "Disponible"
    : "Agotado";

  return (
    <article className="producto-card">

      <h2>{producto.nombre}</h2>

      <p>
        Categoría: {producto.categoria}
      </p>

      <p>
        Precio: ${producto.precio}
      </p>

      <p>
        Stock: {producto.stock}
      </p>

      <strong>{estado}</strong>

    </article>
  );
}

export default ProductoCard;
```

Cambiar:

```
function ProductoCard({ producto }) {
```

por:

```
function ProductoCard({ producto }) {  
  const {  
    nombre,  
    precio,  
    categoria,  
    stock  
  } = producto;
```

Y después:

```
<h2>{nombre}</h2>  
  
<p>Categoría: {categoria}</p>  
  
<p>Precio: ${precio}</p>  
  
<p>Stock: {stock}</p>
```

3. Nuestro primer evento

- Agregar dentro de ProductoCard:

```
const mostrarProducto = () => {  
  alert(`Seleccionaste ${nombre}`);  
};
```

- Y en el JSX:

```
<button onClick={mostrarProducto}>  
  Ver producto  
</button>
```

- Ahora deberían tener:

```
function ProductoCard({ producto }) {  
  const {  
    nombre,
```

```
    precio,
    categoria,
    stock
  } = producto;
  const estado =
    stock > 0
    ? "Disponible"
    : "Agotado";
  const mostrarProducto = () => {
    alert(`Seleccionaste ${nombre}`);
  };
  return (
    <article className="producto-card">
      <h2>{nombre}</h2>
      <p>Categoría: {categoria}</p>
      <p>Precio: ${precio}</p>
      <p>Stock: {stock}</p>
      <strong>{estado}</strong>
      <br />
      <button onClick={mostrarProducto}>
        Ver producto
      </button>
    </article>
  );
}
```

```
export default ProductoCard;
```

Pregunta de control:

¿mostrarProducto es una función de React? No.

Es una **función JavaScript**.

React simplemente la ejecuta cuando ocurre: onClick

4. Botón inteligente

Ahora vamos a aprovechar el stock.

Cambiar el botón por:

```
<button
  onClick={mostrarProducto}
  disabled={stock === 0}
>
  {
    stock > 0
      ? "Ver producto"
      : "Agotado"
  }
</button>
```

Aquí están utilizando tres cosas de ayer:

- condiciones
- ternario
- objetos

dentro de React.

Primer checkpoint Git

- git status
- git add .
- git commit -m "feat: agregar eventos a ProductoCard"

- git push

5. Crear un buscador

En App.jsx:

```
import { useState } from "react";
```

Dentro de App():

```
const [busqueda, setBusqueda] = useState("");
```

Dentro del return de App.jsx, arriba del catálogo:

```
<input  
  type="text"  
  placeholder="Buscar producto..."  
  value={busqueda}  
  onChange={(evento) => {  
    setBusqueda(evento.target.value);  
  }}  
>
```

6. Recuperemos filter()

Ahora llega la razón por la que practicaron filter() ayer.

Antes del return:

```
const productosFiltrados =  
  productos.filter(producto =>  
    producto.nombre  
      .toLowerCase()  
      .includes(  
        busqueda.toLowerCase()  
      )  
  );
```

Y donde anteriormente tenían:

```
productos.map(...)
```

cambiar a:

```
productosFiltrados.map(producto => (  
  <ProductoCard  
    key={producto.id}  
    producto={producto}  
  />  
))
```

Ahora probar:

Mouse

Luego:

Monitor

Luego:

Teclado

La interfaz debe cambiar inmediatamente.

7. Mensaje cuando no existen resultados

Agregar:

```
{  
  productosFiltrados.length === 0  
    ? <p>No se encontraron productos.</p>  
    : null  
}
```

Aquí reaparece:

operador ternario

Checkpoint — 10:00

Segundo commit:

```
git add .
```

```
git commit -m "feat: implementar buscador de productos"
```

```
git push
```

8. Filtrar por categoría

Agregar un segundo estado:

```
const [categoria, setCategoria] =  
  useState("Todas");
```

Crear:

```
<select  
  value={categoria}  
  onChange={(evento) =>  
    setCategoria(evento.target.value)  
  }  
>  
  <option value="Todas">  
    Todas  
  </option>  
  
  <option value="Perifericos">  
    Periféricos  
  </option>  
  
  <option value="Pantallas">  
    Pantallas  
  </option>
```



```
</select>
```

Las categorías deberán corresponder a las que **creaste ayer**.

Ahora modificar el filtro:

```
const productosFiltrados =  
  productos.filter(producto => {  
    const coincideNombre =  
      producto.nombre  
        .toLowerCase()  
        .includes(  
          busqueda.toLowerCase()  
        );  
    const coincideCategoria =  
      categoria === "Todas" ||  
      producto.categoria === categoria;  
    return (  
      coincideNombre &&  
      coincideCategoria  
    );  
  });
```

9. Mostrar solamente disponibles

Agregar:

```
const [soloDisponibles, setSoloDisponibles] =  
  useState(false);
```

Interfaz:

```
<label>
```

```

<input
  type="checkbox"
  checked={soloDisponibles}
  onChange={(evento) =>
    setSoloDisponibles(
      evento.target.checked
    )
  }
/>
    Mostrar únicamente disponibles
  </label>

```

Y agregar al filtro:

```

const coincideStock =
  !soloDisponibles ||
  producto.stock > 0;

```

Finalmente:

```

return (
  coincideNombre &&
  coincideCategoria &&
  coincideStock
);

```

10. Contador dinámico

Ayer calcularon productos disponibles.

Ahora mostrar:

```

<p>
  Productos encontrados:

```

```
{productosFiltrados.length}  
</p>
```

Cuando escriben:

Mouse

el número cambia.

Cuando filtran categoría:

Periféricos

cambia.

11. Una función JavaScript propia

Crear:

```
const formatearPrecio = precio => {  
  return precio.toLocaleString("es-CO");  
};
```

Pasarla a ProductoCard o utilizarla dentro del componente.

Entonces:

```
<p>  
  Precio:  
  ${formatearPrecio(precio)}  
</p>
```

Y aparece:

\$650.000

en vez de:

\$650000

12. Reto libre

Cada aprendiz debe elegir **dos** mejoras.

Opción A

Ordenar productos:

menor precio → mayor precio

Opción B

Ordenar:

mayor precio → menor precio

Opción C

Mostrar:

Producto económico

Producto premium

según el precio.

Opción D

Agregar:

descuento

utilizando map().

Opción E

Mostrar cantidad de productos agotados usando:

filter()

o:

reduce()

Opción F

Botón:

Limpiar filtros

que haga:

setBusqueda("");

setCategoria("Todas");

setSoloDisponibles(false);

13. GitHub final

git status

git add .

git commit -m "feat: agregar filtros interactivos al catalogo"

git push

Y verificar GitHub.

Entregable

1. Repositorio de ayer actualizado

2. Los nuevos commits

3. Aplicación funcionando con:

- productos del taller anterior;
- ProductoCard;
- botón;
- onClick;
- useState;
- buscador;
- filter;
- filtro por categoría;
- filtro por disponibilidad;
- contador de resultados.

4. Explicación corta

Responder:

¿Qué hace useState?

¿Qué diferencia existe entre filter() y map()?

¿Qué ocurre cuando se dispara onChange?

¿Por qué usamos key?

¿Qué información recibe ProductoCard mediante props?

¿Qué diferencia hay entre el proyecto de ayer y el de hoy?